Mastermind Security Review
Final Report

September 1, 2025



Version [1.0]

Email	Telephone
audits@extropy.io	+44 1865338228

Data Classification	Client Confidential
Client Name	Mina Builders Team
Document Title	Mastermind
Document Version	[2.0]
Author	Extropy Audit Team

Contents

1 Executive Summary **1**

2 Audit summary **2**
2.1 Audit scope . . . 2
2.2 Issues Summary . . . 3
2.3 Methodology . . . 4

3 Findings **5**
3.1 [CRITICAL] Malicious CodeMaster can arbitrarily assign referee role and claim game funds 5
3.2 [HIGH] The game is 100 percent solvable with optimal strategies . . . 5
3.3 [HIGH] Game deadlock from turn-based timeouts can lead to incorrect winner . . . 6
3.4 [HIGH] Unrestricted WebSocket Connections . . . 8
3.5 [HIGH] Lack of Input Sanitisation . . . 8
3.6 [HIGH] Game Ending Logic . . . 8
3.7 [HIGH] CodeBreaker unable to claim win on exact final attempt . . . 9
3.8 [HIGH] Proof replay vulnerability due to non-unique ‘solutionHash‘ . . . 10
3.9 [MEDIUM] Improve salt generation security . . . 10
3.10 [MEDIUM] Users can replay a game by moving offchain . . . 11
3.11 [MEDIUM] DOS possible from large malformed proofs . . . 11
3.12 [MEDIUM] Replay attack possible in game cancellation . . . 12
3.13 [MEDIUM] Personally Identifiable Information at risk of being leaked . . . 12
3.14 [LOW] Inconsistent slot verification . . . 12
3.15 [LOW] Missing bit count validation . . . 13
3.16 [LOW] Possibility to erroneously win a game . . . 13
3.17 [LOW] Dependency problem in docker . . . 14
3.18 [LOW] Game ending state varies . . . 14
3.19 [LOW] Wrong error message in on-chain ‘giveClue‘ . . . 15
3.20 [LOW] Nothing prevents playing against oneself . . . 15
3.21 [LOW] Off-chain game start not tied to an on-chain game . . . 15
3.22 [LOW] A game can be hijacked by any codebreaker . . . 16
3.23 [LOW] Game can be won by resubmitting proofs for the same turn . . . 16
3.24 [LOW] A player may collude with the referee for rewards . . . 16
3.25 [LOW] Balance the player stakes . . . 17
3.26 [LOW] Race condition starting game . . . 17
3.27 [LOW] Penalization Logic . . . 17
3.28 [LOW] No fallback function for the Redis server. . . . 18
3.29 [LOW] ZK Program allows proof generation for turns exceeding max attempts . . . 18
3.30 [INFO] Underconstrained code in clues . . . 19
3.31 [INFO] Inconsistency in digit range documentation . . . 19
3.32 [INFO] Logically wrong place for codebreaker check . . . 19
3.33 [INFO] Mismatch in GameState struct default ‘rewardAmount‘ and minimum reward expectation . . . 20
3.34 [INFO] Superfluous ‘super.init()‘ . . . 20
3.35 [INFO] Potentially confusing static method name . . . 20
3.36 [INFO] Redundant logical operation when updating ‘isSolved‘ in ‘giveClue()‘ . . . 21
3.37 [INFO] Lack of progress report on transaction finalisation . . . 21

3.38 [INFO] Inconsistent error messaging for digit validation 22

4 Test Coverage **23**
4.1 Test output 23

A Disclaimers **30**
A.1 Client Confidentiality 30
A.2 Proprietary Information 30

B Automated Tools results **31**
B.1 Burp Suite results 31

C Game Ending Analysis **32**

D Decision Tree **33**

1 | Executive Summary

Extropy was contracted to conduct an initial code review and vulnerability assessment of the Mastermind protocol. The protocol, deployed on the Mina blockchain and built using the o1js framework, implements a verifiable version of the classic Mastermind game leveraging Zero-Knowledge Proofs (ZKPs) to ensure fair play and trustless execution.

Overall, the Mastermind protocol demonstrates a well-structured approach to building a verifiable game on the Mina blockchain showing really good understanding of ZK-specific patterns. The documentation supplied was of high quality and in concert with feedback from the development team allowed us to get up to speed quickly.

In the initial report, several findings were discovered and explained, ranging from critical vulnerabilities to informational observations and design considerations.

Section 3 of this report details each of these findings. Where possible, practical recommendations for their resolution have been provided to enhance the robustness, security, and fairness of the Mastermind protocol.

The logic around movement from off chain to on chain games, penalisation and the criteria for games ending were the areas where most issues were found. In appendix C we give further details of the analysis we performed in this area.

Safe key management is a critical area and though no specific issues were found relating to key management, we want to stress the importance of following best practices and checking that they are followed.

Most the issues have been resolved, and this final report reflects that. We feel that the criteria for game ending logic is still overly complex, and some issues are marked as partially resolved as a result. An issue remains around the ability of a codebreaker to follow an optimal strategy we appreciate that this will be now be monitored as we think that further changes will be necessary.

2 | Audit summary

This audit, conducted from 7th July 2025 to 24th July 2025, employed a comprehensive approach using a manual review method. Our examination aimed to ensure the robustness and security of the Mastermind codebase. The review focused on identifying potential vulnerabilities, logical flaws, and deviations from cryptographic best practices relevant to ZKP applications.

- The code was taken from the following repositories:
  - recursive-mastermind-zkApp
  - recursive-mastermind-stack
- The audit was performed on the following commits:
  - **zkApp repo:** 99d4826 (initial) and a later commit (a581236).
  - **Stack repo:** commit f63dac0.
- **Note:** Within the findings in section 3, any specified commit refers to the initial commit. If no commit is specified, the later commit is implicit.

2.1 | Audit scope

The following source files were audited:

Contract	LoC
stepProgram.ts	218
utils.ts	245
constants.ts	4
index.ts	34
Mastermind.ts	613
In stack repo, in server constants.ts	11
databaseConnections.ts	10
index.ts	242
instruments.ts	14
redisClient.ts	16
services.ts	451
zkAppHandlers.ts	45
routes/gamesRoutes.ts	158
repositories/game.ts	16
models/Game.ts	128
Total	**2205**

2.2 | Issues Summary

ID	Finding	Status
3.1	[CRITICAL] Malicious CodeMaster can arbitrarily assign referee role and claim game funds	Resolved
3.2	[HIGH] The game is 100 percent solvable with optimal strategies	Unresolved
3.3	[HIGH] Game deadlock from turn-based timeouts can lead to incorrect winner	Unresolved
3.4	[HIGH] Unrestricted WebSocket Connections	Resolved
3.5	[HIGH] Lack of Input Sanitisation	Resolved
3.6	[HIGH] Game Ending Logic	Resolved
3.7	[HIGH] CodeBreaker unable to claim win on exact final attempt	Unresolved
3.8	[HIGH] Proof replay vulnerability due to non-unique ‘solutionHash‘	Resolved
3.9	[MEDIUM] Improve salt generation security	Resolved
3.10	[MEDIUM] Users can replay a game by moving offchain	Unresolved
3.11	[MEDIUM] DOS possible from large malformed proofs	Resolved
3.12	[MEDIUM] Replay attack possible in game cancellation	Resolved
3.13	[MEDIUM] Personally Identifiable Information at risk of being leaked	Resolved
3.14	[LOW] Inconsistent slot verification	Unresolved
3.15	[LOW] Missing bit count validation	Resolved
3.16	[LOW] Possibility to erroneously win a game	Unresolved
3.17	[LOW] Dependency problem in docker	Resolved
3.18	[LOW] Game ending state varies	Unresolved
3.19	[LOW] Wrong error message in on-chain ‘giveClue‘	Resolved
3.20	[LOW] Nothing prevents playing against oneself	Resolved
3.21	[LOW] Off-chain game start not tied to an on-chain game	Resolved
3.22	[LOW] A game can be hijacked by any codebreaker	Resolved
3.23	[LOW] Game can be won by resubmitting proofs for the same turn	Resolved
3.24	[LOW] A player may collude with the referee for rewards	Resolved
3.25	[LOW] Balance the player stakes	Unresolved
3.26	[LOW] Race condition starting game	Resolved
3.27	[LOW] Penalization Logic	Resolved
3.28	[LOW] No fallback function for the Redis server	Resolved
3.29	[LOW] ZK Program allows proof generation for turns exceeding max attempts	Resolved
3.30	[INFO] Underconstrained code in clues	Resolved
3.31	[INFO] Inconsistency in digit range documentation	Unresolved
3.32	[INFO] Logically wrong place for codebreaker check	Resolved
3.33	[INFO] Mismatch in GameState struct default ‘rewardAmount‘ and minimum reward expectation	Unresolved
3.34	[INFO] Superfluous ‘super.init()‘	Resolved
3.35	[INFO] Potentially confusing static method name	Resolved
3.36	[INFO] Redundant logical operation when updating ‘isSolved‘ in ‘giveClue()‘	Resolved
3.37	[INFO] Lack of progress report on transaction finalisation	Unresolved
3.38	[INFO] Inconsistent error messaging for digit validation	Unresolved

2.3 | Methodology

2.3.1 | Risk Rating

The risk rating given for issues follows the standard approach of the OWASP Foundation. We combine two factors :

- Likelihood of exploit
- Impact of Exploit

The Categories we use are *Critical*, *High*, *Medium*, *Low* and *Informational* These categories may not align with the categories used by other companies.
The informational category is used to contain suggestions for optimisation (where this is not seen as causing significant impact), or for alternative design or best practices.

2.3.2 | Approach

The project was assessed mainly by code inspection, with auditors working independently or together, looking for possible exploits. Tests were written were possible to validate the issues found.

3 | Findings

3.1 | [CRITICAL] Malicious CodeMaster can arbitrarily assign referee role and claim game funds

- **Location(s):** Mastermind.ts#156
- **Commit:** 99d4826[1]
- **Description:** Within `Mastermind.ts`, the `initGame()` function allows the codeMaster (the sender of the initGame transaction) to specify any `refereePubKey` as an input parameter. The contract then sets `this.refereeId` based on this provided `refereePubKey`. A critical vulnerability arises because a malicious codeMaster can set a public key accessible by them during game initialization.

  Subsequently, the `forfeitWin()` method (which correctly enforces that only the referee can call it via `refereeId.assertEquals(...)`) becomes callable by this self-assigned codeMaster/referee. This mechanism could be exploited by the codeMaster to unilaterally call `forfeitWin()` and claim the `rewardAmount` (which originates from the codeMaster's initial deposit and potentially further deposits from the codeBreaker later in the game), thereby allowing the codeMaster to steal codeBreaker deposits or prevent the legitimate conclusion of the game, bypassing standard game progression and fairness.

```
@method async forfeitWin(playerPubKey: PublicKey) {
  const refereeId = this.refereeId.getAndRequireEquals();
  ...
  refereeId.assertEquals(
    Poseidon.hash(this.sender.getAndRequireSignature().toFields()),
    'You are not the referee of this game!'
  );
```

- **Recommendation:**

1. Consider implementing robust role separation and validation to prevent an arbitrary address being assigned as the referee. A whitelist might be considered.
2. Add functionality to increase the turn-based timeout upon proof submission (as long as the proof progresses the game)

- **Status:** Resolved.

3.2 | [HIGH] The game is 100 percent solvable with optimal strategies

- **Location(s):** N/A
- **Description:** Like the traditional mastermind game, also this variant can most likely be solved within the given guesses with 100 percent certainty. The codebreaker only needs to use optimal strategies.

  What makes this dangerous is the included monetary incentive. Without it, there is no incentive to force a win. If the reward is good enough (or if there are enough games with a small incentive), codebreakers will come up with optimal strategies, guaranteeing them a win.

  The traditional mastermind game can be won with 5 or less guesses: https://en.wikipedia.org/wiki/Mastermind_(boa guess_algorithm

[1] https://github.com/mina-builders-team/recursive-mastermind-zkApp/tree/99d4826994a7ddc9e7f3f5945b079e7facae6390

- **Recommendation:** Decrease the number of turns, so that luck plays a part and a win by the codebreaker is not guaranteed.
- **Status:** Unresolved.

Team response :
Since the clues are unordered and there are 8 possible digits, the maxAttempt value was determined accordingly. Changing this number can be done simply by modifying a constant (link to code[2]), so we are leaving it as is for now, because we want to see how fair the game is during the Devnet and early Mainnet phase. Accordingly, we can make the game more difficult if it requires.
Auditors' response :
We have moved the severity down to HIGH as the team has a plan to mitigate this problem when it occurs.

3.3 | [HIGH] Game deadlock from turn-based timeouts can lead to incorrect winner

- **Location(s):** Mastermind.ts#371-373
- **Description:** The contract's timeout mechanism is inconsistent. While individual moves (`makeGuess`, `giveClue`) correctly check for per-turn timeouts, the `claimReward` function only checks against the game's final deadline (`finalizeSlot`). This discrepancy allows a malicious player to intentionally time out on their turn, causing a deadlock that freezes the game state and locks both players funds until the final deadline is reached.

Current implementation in `makeGuess` and `giveClue`:

```
lastPlayedSlot
      .add(PER_TURN_GAME_DURATION)
      .assertGreaterThanOrEqual(
        currentSlot,
        'You have passed the time limit to make a guess!'
      );
```

`claimReward` should be the remedy for timeouts but it only checks if the overall game duration has expired not each turn:

```
let isFinalized = this.network.globalSlotSinceGenesis
      .getAndRequireEquals()
      .greaterThanOrEqual(finalizeSlot);
```

Once the game reaches finalisation either player can successfully claim because of these missing checks creating a race condition. `-codeBreaker` can claim because `isCodeMasterLeft` becomes true (`codeMaster` failed to act on their turn, and the game is finalised). `-codeMaster` can also claim because `const codeMasterWinByFinalize = isSolved.not().and(isFinalized);` is true.

- **Recommendation:** Consider changing the winning logic to something like:

```
...
    isCodeMaster
      .or(isCodeBreaker)
      .assertTrue('You are not the codeMaster or codeBreaker of this game!');
```

[2] https://github.com/mina-builders-team/recursive-mastermind-zkApp/blob/42c57fbfd69115eec287c6cba594297fae7e37ab/src/constants.ts#L5-L6

```
    // Unified Timeout Logic
    const currentSlot = this.network.globalSlotSinceGenesis.getAndRequireEquals();
    const isGloballyFinalized = currentSlot.greaterThanOrEqual(finalizeSlot);
    const isTurnTimedOut = lastPlayedSlot
      .add(PER_TURN_GAME_DURATION)
      .lessThan(currentSlot);
    const anyTimeout = isGloballyFinalized.or(isTurnTimedOut);

    // Define Mutually Exclusive Win Conditions
    const isBreakersTurnToPlay = turnCount.value.isOdd();

    const codeBreakerIsWinner = isSolved.or(
      anyTimeout.and(isBreakersTurnToPlay.not())
    );

    const codeMasterIsWinner = isSolved.not().and(
      turnCount
        .greaterThan(MAX_ATTEMPTS * 2)
        .or(anyTimeout.and(isBreakersTurnToPlay))
    );

    const callerIsWinner = Provable.if(
      isCodeBreaker,
      codeBreakerIsWinner,
      codeMasterIsWinner
    );
    callerIsWinner.assertTrue('You are not the winner of this game!');
...
```

This allows a player to call `claimReward` if any timeout has occurred, preventing a deadlock. Also, clearer individual winning conditions for each player.

- **Status:** Unresolved

Team Response:
"The contract's timeout mechanism is actually quite consistent in itself. There is no scenario where players can deadlock the game by using the onchain giveClue and makeGuess methods. The reasons for the difference in timeout checks between these methods and the claimReward method can be explained as follows: The game was mainly designed to be played using offchain methods, and the onchain methods were added as a fallback in case of issues with the relayer. Since it's not possible to check whether the relayer is active onchain, moves played via onchain methods are allowed to be overwritten by a gameProof containing a longer move chain. In this way, we prevent malicious players from forking the game incorrectly using only their own single-method moves. However, since there isn't a third trusted party here either, we provide players a limited time to make their move other than the finalizeSlot. If a player doesn't make their move within their allocated time, they can deadlock the game for themselves and lose by being unable to play until the game completes. This is where the offchain proof overwrite mechanism comes into play. If the game is progressing smoothly offchain and one player tries to lock or make the other player lose by moving the game onchain, the proof overwrites the current game state with their own state, preventing such deadlock. Both mechanisms are calculated to conclude at the same finalizeSlot, ensuring that either approach works for the whole game."

Auditors' Response
We still feel that this is a possibility, the proof will not overwrite what is on chain as no turns have been completed

3.4 | [HIGH] Unrestricted WebSocket Connections

- **Location(s):** index.ts#
- **Description:** The WebSocket server initialisation and connection handling in src/index.ts does not implement any form of rate-limiting, authentication, or connection limits. This allows an attacker to open an unlimited number of WebSocket connections, consuming server memory and CPU resources. A large number of concurrent connections could exhaust the server's resources, leading to a denial-of-service condition that would prevent legitimate players from connecting to the game service.
-
- **Recommendation:** Implement rate limiting and connection limits
- **Status:** Resolved.

3.5 | [HIGH] Lack of Input Sanitisation

- **Location(s):** gamesRoute.ts , repositories/game.ts, all functions interacting with the database
- **Description:** The application uses express.json() to parse request bodies in src/index.ts.

  However, there is no explicit sanitization of the data before it's used in database queries. For example, in src/repositories/game.ts, the createOrUpdateGame function takes a gameData object and uses it directly in a findOneAndUpdate query.

  Without proper sanitisation, this could be vulnerable to NoSQL injection attacks where an attacker could craft a malicious gameData object to modify the query and gain unauthorised access or corrupt the database.
- **Recommendation:** Sanitise all possible inputs
- **Status:** Resolved.

3.6 | [HIGH] Game Ending Logic

- **Location(s):** N/A
- **Description:** The logic needed to determine the end of the game is understandably complex, especially since we have to account for server failures and turn timeouts.

The design currently depends mainly on 3 variables in submitGameProof and 5 variables in claimReward , but this leads to fragility and the possibility that these variables could be out of sync in some edge cases.
The variables are flags to indicate

- "You are codemaster"
- "It is the codemaster's turn"
- "A solution has been found"

- "There is still time left"
- "Less than 16 turns used"
- **Recommendation:** Consider changing to a state machine design where a single variable will determine the state of the game, and the transitions are more explicit and easier to reason about.
- **Status:** Unresolved.

Team response :
In both methods, ending the game is governed by a set of checks, and since some of these checks depend dynamically on the inputs or the slot number, handling them with a single state variable is not feasible. Moreover, breaking down the checks into smaller parts and combining them in this way allows us to write code that is self-explanatory and easier to read and debug after.

3.7 | [HIGH] CodeBreaker unable to claim win on exact final attempt

- **Location(s):** Mastermind.ts#303
- **Commit:** 99d4826[3]
- **Description:** Within `Mastermind.ts`, the game logic defines `turnCount` such that when the game is created its value is 1, then all CodeBreaker guesses are associated with odd numbers (1, 3, ..., up to MAX_ATTEMPTS * 2 = 14), and CodeMaster clues are associated with even numbers (2, 4, ..., up to 14). When the CodeBreaker successfully guesses the combination on the 7th (and final) attempt, the `giveClue()` function correctly sets `isSolved` to true and updates the `turnCount` to 15 (reflecting the CodeMaster's action of giving the final clue).

  However, in the `submitGameProof()` function, the `maxAttemptsExceeded` variable is calculated as

  ```
  const maxAttemptsExceeded = proof.publicOutput.turnCount.greaterThan(
    MAX_ATTEMPTS * 2
  );
  ```

  Thus when the game is solved on the 7th attempt, `proof.publicOutput.turnCount` will be 15. Consequently, 15 ¿ 14 evaluates to true, making `maxAttemptsExceeded` also true. This causes a critical issue for any subsequent win condition that relies on it, such as

  ```
  isSolved = clue.isSolved().and(maxAttemptsExceeded.not());
  ```

  Since `maxAttemptsExceeded.not()` would evaluate to false when turnCount is 15, the overall win condition fails, even though `isSolved` is true. This prevents the legitimate CodeBreaker from claiming their deserved reward when they solve the combination exactly on the last allowed attempt.
- **Recommendation:** Consider adjusting the `maxAttemptsExceeded` logic in `submitGameProof()` to accurately account for a win on the final attempt by checking `proof.publicOutput.turnCount` against `(MAX_ATTEMPTS * 2) + 1` to define the actual point where attempts are fully exhausted.
- **Status:** Fixed.

[3] https://github.com/mina-builders-team/recursive-mastermind-zkApp/tree/99d4826994a7ddc9e7f3f5945b079e7facae6390

3.8 | [HIGH] Proof replay vulnerability due to non-unique 'solutionHash'

- **Location(s):** Mastermind.ts#269-351
- **Commit:** 99d4826[4]
- **Description:** Within `Mastermind.ts`, the `submitGameProof()` function is responsible for verifying the state transitions of the game by comparing the following on chain fields against the proof's public output: `CodeBreakerId`, `CodeMasterId`, `SolutionHash`, and `turnCount` . Crucially, the proof's internal `packedGuessHistory` and `packedClueHistory` are not explicitly verified against the on-chain state, meaning the actual sequence of guesses and clues is not directly checked during proof submission.

  A significant vulnerability arises because the `solutionHash` (calculated as the Poseidon Hash of `secretCombination.digits` and `salt` does not incorporate a unique identifier specific to the current game instance, such as the contract's address or a unique game nonce.

  This creates a replay attack vector:

  1. A malicious codeMaster could play a game (Game 1) with a codeBreaker.
  2. If the codeMaster then initiates a new game (Game 2) with the same codeBreaker and, critically, reuses the exact same secret combination and salt as in Game 1, both games would share an identical solutionHash.
  3. The codeMaster could then save a proof from an intermediate stage of Game 1 (e.g., a proof corresponding to turnCount = 5).
  4. In Game 2, after the codeBreaker has made only a few initial guesses (e.g., turnCount = 2), the malicious codeMaster could submit the saved proof from Game 1.
  5. Since the codeMasterId, codeBreakerId, and solutionHash would match, and the proof's turnCount (5) is greater than Game 2's current turnCount (2), the old proof from Game 1 would successfully validate Game 2's state update.
  6. This exploit would arbitrarily advance Game 2's turnCount to 5, potentially bypassing the codeBreaker's remaining attempts, forcing a premature game end, or leading to an unjust forfeiture, ultimately allowing the codeMaster to win unfairly or steal the codeBreaker's stake.

- **Recommendation:** Consider integrating a robust, unique game identifier into the `solutionHash` calculation to effectively prevent proof replay attacks across different game instances. A strong solution would involve including the contract's own address or a unique game nonce as an additional input to the `Poseidon.hash()` function when computing `solutionHash` in `initGame()`. This ensures that the solutionHash is cryptographically bound to a specific game instance, rendering proofs from other games invalid for replay.
- **Status:** Resolved.

3.9 | [MEDIUM] Improve salt generation security

- **Location(s):** stack/utils.ts#161-168
- **Description:** In the `generateRandomSalt` function, `Math.random()` is used. However, in testing `Field.random()` is used to generate the random salt.

```
export function generateRandomSalt(length = 20): string {
  const chars = '123456789';
```

[4] https://github.com/mina-builders-team/recursive-mastermind-zkApp/tree/99d4826994a7ddc9e7f3f5945b079e7facae6390

```
  let randomSalt = '';
  for (let i = 0; i < length; i++) {
    randomSalt += chars.charAt(Math.floor(Math.random() * chars.length));
  }
  return randomSalt;
}
```

In testing:

```
codeMasterSalt = Field.random();
```

`Math.random()` is psuedo-random and often generates its seed from a deterministic source, such as the current system time. Whereas, `Field.random()` generates its seed from multiple chaotic physical sources (computationally infeasible to predict).

- **Recommendation:** Consider changing the salt generation to use `Field.random()`, making it cryptographically secure.

```
export function generateRandomSalt(): string {
  const randomSalt = Field.random();
  return randomSalt.toString(); // Convert to string for UI compatibility
}
```

- **Status:** Resolved.

3.10 | [MEDIUM] Users can replay a game by moving offchain

- **Location(s):** N/A
- **Description:** A game can be restarted from an earlier point if switching an on-chain game back to the off-chain proofs.

Scenario:

1. A game is started normally
2. A few turns are played off-chain
3. Server goes down for a bit and the game moves on-chain
4. A few turns are played on-chain
5. The codebreaker tricks the other player to continue their game off-chain. This could be something like "let's avoid gas fees by going back to off-chain gaming." The off-chain game is at an earlier position and therefore gives an unfair advantage to the codebreaker.

The game still remains within the original `finalizeSlot` so it is a valid game.

- **Recommendation:** Add logic to disable `submitGameProof` once a game has transitioned to an on-chain game.
- **Status:** Unresolved.

3.11 | [MEDIUM] DOS possible from large malformed proofs

- **Location(s):** services.ts#75

- **Description:** In the handleProof function, a zkProof is received as a JSON string and then parsed and verified. There are no checks on the size of the zkProof string or the complexity of the proof before it is processed. An attacker could send a very large or computationally expensive proof, causing the server to consume significant resources while trying to verify it. This could slow down or crash the server, impacting all ongoing games.
- **Recommendation:** Consider rejecting inputs over a certain size.
- **Status:** Resolved.

3.12 | [MEDIUM] Replay attack possible in game cancellation

- **Location(s):** gamesRoute.ts, POST /cancel/:id route
- **Description:** The game cancellation endpoint correctly verifies a signature to authorize the action. However, there is no mechanism to prevent a replay attack.

  An attacker who intercepts a valid signed cancellation request could replay it multiple times.

  While the game status check ([GameStatus.ACTIVE, GameStatus.PENDING, GameStatus.CANCELLED].includes(game provides some protection, it is still a flawed design.
- **Recommendation:** Consider using a nonce or a unique identifier in the signed message to ensure that each signature can only be used once.
- **Status:** Resolved.

3.13 | [MEDIUM] Personally Identifiable Information at risk of being leaked

- **Location(s):** instruments.ts#9 also used in services.ts
- **Description:** The setup of SENTRY_DSN includes a flag `sendDefaultPii` which has been set to true. This means that in event of an error sensitive information could be sent to Sentry. This could further have GDPR implications.
- **Recommendation:** Set the flag to false to remove PII.
- **Status:** Resolved.

3.14 | [LOW] Inconsistent slot verification

- **Location(s):** zkApp/Mastermind.ts#121-125, 234-238
- **Description:** Sometimes the received globalSlotSinceGenesis is verified oddly (like seen in the following snippet) and sometimes it's verified with simple `getAndRequireEquals()`. There is no reason to use the elaborate method.

```
const currentSlot = this.network.globalSlotSinceGenesis.get();
this.network.globalSlotSinceGenesis.requireBetween(
  currentSlot,
  currentSlot.add(UInt32.from(1))
);
```

- **Recommendation:** Consider switching all instances of `globalSlotSinceGenesis` to utilize `globalSlotSinceGenesi`
- **Status:** Resolved.

3.15 | [LOW] Missing bit count validation

- **Location(s):** zkApp/utils.ts#229
- **Description:** Throughout the code explicit bit counts are used for clarity and validation. A few examples:

```
static decompress(compressedCombination: Field) {
    const bits = compressedCombination.toBits(12);
```

And:

```
static decompress(compressedClue: Field) {
    const bits = compressedClue.toBits(6);
```

However, in `GameState:unpack` it is not specified.

```
static unpack(serializedState: Field) {
    const bits = serializedState.toBits();
```

- **Recommendation:** Stay consistent and match the pattern used elsewhere. It is clearer and provides implicit validation of bit length.

```
static unpack(serializedState: Field) {
    const bits = serializedState.toBits(137);
```

- **Status:** Resolved.

3.16 | [LOW] Possibility to erroneously win a game

- **Location(s):** N/A
- **Description:** It is possible for either party to claim the reward without playing the game till the end if the server/UI is malfunctioning. Flow:

1. Players start a game normally
2. They play a bit but don't finalize the game.
3. The non-malicious player does not update the on-chain state
4. The malicious player (can be either one) waits for the finalization deadline to pass and claims the reward with their latest proof.

Normally, this is mitigated by two factors:

1. The server should interfere before the finalization timeout and declare one player the winner
2. If the server is down, the UI should prompt both players to post their latest proof on-chain

However, it's possible that the UI and server do not function properly, in which case a game can be won erroneously.

- **Recommendation:** Make this possibility clear in the documentation to advise the players.
- **Status:** Unresolved.

This still has a dependency on the server working correctly.

3.17 | [LOW] Dependency problem in docker

- **Location(s):** dockerfile.dev
- **Description:** Missing vite dependency in docker
- **Recommendation:** Change the order in the dockerfile to

```
FROM node:20-slim
WORKDIR /app

COPY ui/ .

RUN yarn install

CMD ["yarn", "run", "dev"]
```

- **Status:** Resolved.

3.18 | [LOW] Game ending state varies

- **Location(s):** zkApp/Mastermind.ts#110
- **Description:** Game end is currently denoted and detected by various means. For example the field `finalizeslot` is sometimes set to zero (in `claimReward`) and sometimes kept intact (in `forfeitWin`). There is no universal way to simply detect whether a certain game has ended and reward has been distributed.

In `assertNotFinalized` one bug-prone mechanism is used (it even requires a dedicated comment to understand the assertion):

```
// When reward claimed, finalizeSlot is set to 0, but codeBreakerId is not
finalizeSlot
  .equals(UInt32.zero)
  .and(codeBreakerId.equals(Field.from(0)).not())
  .assertFalse(
    'The game has already been finalized and the reward has been claimed!'
  );
```

This ambiguous finalization style causes weird errors. For example, after a referee has executed `forfeitWin`, making a new guess is only prevented by an error "You have passed the time limit to make a guess!" because the `lastPlayedSlot` has been set to zero. Whereas, the previous check (`assertNotFinalized`) should have already caught this.

- **Recommendation:** Consider adding a new variable in `GameState` that clearly states when a game has ended and reward has been distributed. This can then be used universally to detect a finalized game.
- **Status:** Partially Resolved.

Team response :
"It was initially thought that there was no need to handle this situation separately after forfeitWin is called, since the reward pool is emptied and the game is essentially over. However, adding this for clarity is a good idea. Creating a variable to cover all possible finalized states of the game didn't seem very

feasible because slot time is dynamic and can change at any moment. Instead, assertions were added within the assertNotFinalized method to specifically check if forfeitWin has been called, returning an appropriate error message in such cases."
Auditors' response :
We accept that this may resolve some of the effects, but there is an underlying problem that the game ending logic is too complex.

3.19 | [LOW] Wrong error message in on-chain 'giveClue'

- **Location(s):** zkApp/Mastermind.ts#599
- **Description:** In the `giveClue` function, the current on-chain time slot is compared against a deadline. If the check fails, an error message is thrown. This error message is wrong, since it talks about making a guess.

```
lastPlayedSlot
  .add(PER_TURN_GAME_DURATION)
  .assertGreaterThanOrEqual(
    currentSlot,
    'You have passed the time limit to make a guess!'
  );
```

- **Recommendation:** Consider fixing the error message to refer to clue giving.
- **Status:** Resolved.

3.20 | [LOW] Nothing prevents playing against oneself

- **Location(s):** zkapp/src/Mastermind.ts#L211
- **Description:** The contract allows the same player to be both codeMaster and codeBreaker because there is no check preventing the codeMaster from calling `acceptGame()`. This can lead to artificially inflating win ratio of some players. While not a direct theft vector, this breaks game fairness and enables reward-gaming exploits.
- **Recommendation:** Consider adding assertions to prevent codeMaster from accepting their own game.
- **Status:** Resolved.

3.21 | [LOW] Off-chain game start not tied to an on-chain game

- **Location(s):** zkApp/stepProgram.ts#58
- **Description:** A typical game is first started on-chain and then moved off-chain, by calling the `createGame` functionality in `stepProgram.ts`. However, there are no guarantees that the off-chain game uses the same secret combination as the one on-chain.

Finalizing a game that uses a different secret combination is not possible on-chain, since the game's data is different, but this can cause users to uselessly play games that can't be finalized.
Furthermore, an on-chain version of the game does not need to exist at all - the used `contractAddress` can be arbitrary.

- **Recommendation:**
- **Status:** Resolved.

3.22 | [LOW] A game can be hijacked by any codebreaker

- **Location(s):** zkApp/stepProgram.ts#126
- **Description:** In the `makeGuess` function in `stepProgram.ts`, the codeBreaker's identity is checked by:

```
previousClue.publicOutput.codeBreakerId
  .equals(computedCodebreakerId)
  .or(turnCount.equals(1))
  .assertTrue('You are not the codeBreaker of this game!');
```

  Due to this condition, on the first turn, anyone can become the codebreaker of that game - regardless of who is set as the codebreaker in the on-chain state.

  The potential risk is small, due to:

  1. A game with the wrong codebreaker can't be finalized on-chain.
  2. The initial proof (that is required for `makeGuess`) should not be publicly available, it's only sent to the right player.
  3. The codemaster (or the server) may decide to not utilize/propagate proofs from the wrong codebreaker.

- **Recommendation:** Consider establishing the codebreaker's identity already in the `createGame` function in `stepProgram.ts`, since that's already known at that stage.
- **Status:** Resolved.

3.23 | [LOW] Game can be won by resubmitting proofs for the same turn

- **Location(s):** N/A
- **Description:** In an off-chain game, if a codemaster simply responds to given proofs, a codebreaker can keep making new proofs for new guesses for a single turn. This way the codebreaker can try as many time as they want, guaranteeing victory.

The risk of this happening in reality is small, since the server checks for the right turn succession.

- **Recommendation:** Add a rule to the documentation to advise the codemaster to accept the first proof submitted and ignore later proofs.
- **Status:** Resolved.

3.24 | [LOW] A player may collude with the referee for rewards

- **Location(s):** N/A
- **Description:** A player may reach out to the referee (or to the entity controlling the referee) and collude with them to cause an unwarranted `forfeitWin`, for rewards.
- **Recommendation:** Consider making the referee's actions more public - for example a section in the UI showing when a referee has called `forfeitWin` on a game and why.
- **Status:** Resolved.

3.25 | [LOW] Balance the player stakes

- **Location(s):** N/A
- **Description:** A game requires both players to enter an equal stake. However, deploying the Mina contract requires 1 MINA that is paid by the codemaster and is not part of the eventual rewards.

This imbalance makes the game financially imbalanced, since the aim is to have both codebreaker and codemaster win about 50% of their games.

- **Recommendation:** Consider requiring the codebreaker to pay 1 MINA more for the reward, to balance the game.
- **Status:** Unresolved.

Team response
"For sustainably fund our transaction and server expenses, we added a RefereeFee to the Mastermind contract that receives 3 Mina per game. To balance this cost between players, we configured the fee as 1 Mina for the Code Master and 2 Mina for the Code Breaker. This way, each party ends up paying an additional 2 Mina besides the rewards."
Auditors' response
This changes the economics considerably. Aborting a game before it has started doesn't give a refund, which is a disincentive to the codemaster. In addition the referee has even more incentive to force stop games maliciously

3.26 | [LOW] Race condition starting game

- **Location(s):** services.ts#218
- **Description:** In the handleGameStart function, the code first fetches the game from the database and checks if a codeBreaker has been assigned. If not, it proceeds to check the on-chain state and then updates the game. There is a time window between the initial check and the final update.

  An attacker could potentially send multiple startGame requests for the same game, and if timed correctly, could create a race condition where the game is started multiple times or with inconsistent state.
- **Recommendation:** Consider adding a locking mechanism
- **Status:** Resolved.

3.27 | [LOW] Penalization Logic

- **Location(s):** service.ts
- **Description:** The checkForPenalization function uses a fixed TURN_DURATION of 2.5 minutes to determine if a player should be penalized.

  This does not account for network latency or other potential delays outside of the player's control. A player could be unfairly penalized.

  Additionally, a malicious player could repeatedly call the penalize action, triggering multiple checkForPenalization checks, which could add unnecessary load to the server.
- **Recommendation:** Consider increasing the time it takes to cause a player to timeout.
- **Status:** Resolved.

3.28 | [LOW] No fallback function for the Redis server.

- **Location(s):** Server/src/RedisClient.ts#14
- **Description:** in the redisClient.ts file. The redis Client is initialized and connected however the connection lacks sufficient error handling or a fallback function.

```
client.on('error', (err) => console.error('Redis Client Error', err));
client.connect()
```

in the case of utilizing the server to store game-related data, having a fallback function would be beneficial to prevent data loss or any potential DOS attacks.

- **Recommendation:** implement proper error handling and consider adding a try/catch block
- **Status:** Resolved.

3.29 | [LOW] ZK Program allows proof generation for turns exceeding max attempts

- **Location(s):** stepProgram.ts#118-120 . stepProgram.ts#185-188
- **Commit:** 99d4826[5]
- **Description:** Within `stepProgram.ts`, the `makeGuess()` and `giveClue()` methods are central to advancing the game's state and generating recursive proofs. While these methods correctly enforce the strict odd/even turn order, they currently lack a crucial upper bound check on the turnCount value obtained from the `previousProof.publicOutput`. This omission allows for the generation of valid ZK proofs for game turns that extend beyond the MAX_ATTEMPTS limit defined by the game's rules. Indeed in `makeGuess()`, the following assertion

```
turnCount
      .isEven()
      .assertFalse('Please wait for the codeMaster to give you a clue!');
```

allows a CodeBreaker to make a guess even if the turnCount from the previous proof is, for example, 15 (reaching the value 16 after the execution which is beyond MAX_ATTEMPTS * 2 + 1 = 15).
Similarly, in `giveClue()`, the following assertion

```
turnCount
      .isEven()
      .and(turnCount.equals(0).not())
      .assertTrue('Please wait for the codeBreaker to make a guess!');
```

allows a CodeMaster to give a clue even if the turnCount from the previous proof is, for example, 16 (representing an 8th guess attempt).
This unchecked progression allows players to generate recursive proofs for an indefinite number of turns, well past the intended `MAX_ATTEMPTS`. While the on-chain `submitGameProof()` function may contain its own checks for MAX_ATTEMPTS, failing to enforce this rule within the core ZK program can lead to the creation of invalid game state proofs.

- **Recommendation:** Consider adding explicit upper bound assertions on `turnCount` within both the `makeGuess()` and `giveClue()` methods in `stepProgram.ts` to enforce the `MAX_ATTEMPTS` rule directly at the proof generation level.
- **Status:** Resolved

[5] https://github.com/mina-builders-team/recursive-mastermind-zkApp/tree/99d4826994a7ddc9e7f3f5945b079e7facae6390

3.30 | [INFO] Underconstrained code in clues

- **Location(s):** zkApp/utils.ts#148
- **Description:** Calculating clues utilizes traditional `if` branching which leads to unprovable code. However, in this case, the unprovable code does not cause issues. Still, it's best to not write unprovable code.

```
for (let i = 0; i < 4; i++) {
  for (let j = 0; j < 4; j++) {
    const isEqual = guess[i].equals(solution[j]).toField();
    if (i === j) {
      hits = hits.add(isEqual);
    } else {
      blows = blows.add(isEqual);
    }
  }
}
```

- **Recommendation:** Consider removing the traditional branching and utilizing `Provable.if` instead.
- **Status:** Resolved.

3.31 | [INFO] Inconsistency in digit range documentation

- **Location(s):** zkApp/utils.ts#8
- **Description:** Within `utils.ts` the comments for the `Combination` class state:

```
@param digits - An array of 4 unique digits between 1 and 7.
```

This suggests the allowed digits are 1,2,3,4,5,6,7 (7 digits). However in the documentation, UI and in the `validate()` logic, 0,1,2,3,4,5,6,7 (8 digits) are allowed.

- **Recommendation:** Consider changing the comment to avoid confusion for anyone looking at the code.
- **Status:** Resolved.

3.32 | [INFO] Logically wrong place for codebreaker check

- **Location(s):** zkApp/Mastermind.ts#117
- **Description:** The `assertNotFinalized` function has an assertion for the codeBreaker to be set:

```
codeBreakerId
      .equals(Field.from(0))
      .assertFalse('The game has not been accepted by the codeBreaker yet!');
```

The function `assertNotFinalized` is utilized in three places:

1. `submitGameProof` function
2. `makeGuess` function
3. `giveClue` function

However, the first two functions already verify the codeBreaker identity (dfferently). Therefore, this logic is only useful in the `giveClue` function.
Furthermore, a function called `assertNotFinalized` does not sound like it should be doing assertions on the game's acceptance.

- **Recommendation:** Consider moving the codebreaker assertion to the `giveClue` function.
- **Status:** Resolved.

3.33 | [INFO] Mismatch in GameState struct default 'rewardAmount' and minimum reward expectation

- **Location(s):** utils.ts#206
- **Commit:** 99d4826[6]
- **Description:** Within `utils.ts`, the GameState struct's default static property initializes `rewardAmount` to `UInt64.from(1e9)`. However, documentation or implicit game rules suggest that the minimum reward amount for each game is intended to be at least `1e10` namely 10 MINA. This creates a discrepancy between the default value set in the code and the expected minimum value.

```
static default = new this({
  rewardAmount: UInt64.from(1e9),
  finalizeSlot: UInt32.from(0),
  lastPlayedSlot: UInt32.from(0),
  turnCount: UInt8.from(0),
  isSolved: Bool(false),
});
```

- **Recommendation:** Consider changing the `rewardAmount` default value in `GameState.default` from `UInt64.from(1e9)` to `UInt64.from(1e10)`
- **Status:** Resolved.

3.34 | [INFO] Superfluous 'super.init()'

- **Location(s):** zkApp/Mastermind.ts#165
- **Description:** The function `initGame` includes a call `super.init()`. However, the contract does not have a custom `init` function, therefore the default one is used. There is no need to call `super.init()` unless one wants to override the default `init` constructor with custom logic.
- **Recommendation:** Consider removing the call `super.init()`.
- **Status:** Resolved.

3.35 | [INFO] Potentially confusing static method name

- **Location(s):** zkApp/utils.ts#141
- **Description:** `giveClue` is the name of the method that generates the clue, doesn't actually give the clue.

[6] https://github.com/mina-builders-team/recursive-mastermind-zkApp/tree/99d4826994a7ddc9e7f3f5945b079e7facae6390

```
static giveClue(guess: Field[], solution: Field[]) {
    let hits = Field(0);
    let blows = Field(0);

    for (let i = 0; i < 4; i++) {
      for (let j = 0; j < 4; j++) {
        const isEqual = guess[i].equals(solution[j]).toField();
        if (i === j) {
          hits = hits.add(isEqual);
        } else {
          blows = blows.add(isEqual);
        }
      }
    }
    return new this({ hits, blows });
  }
```

- **Recommendation:** Consider renaming it to `generateClue` to avoid confusion.
- **Status:** Resolved.

3.36 | [INFO] Redundant logical operation when updating 'isSolved' in 'giveClue()'

- **Location(s):** Mastermind.ts#602
- **Commit:** 99d4826[7]
- **Description:** Within `Mastermind.ts`, the `giveClue()` method updates within the compressed game state the `isSolved` variable using the following line:

  ```
  isSolved = isSolved.or(clue.isSolved());
  ```

  However, prior to this line, the `this.assertNotFinalized(finalizeSlot, isSolved)` method is invoked: internally it asserts that the game is not yet solved, thus guaranteeing that the `isSolved` variable unpacked from `this.compressedState` and passed as the second function argument is `Bool(false)`. Consequently, the `isSolved.or(clue.isSolved())` operation becomes redundant, as `Bool(false).or(X)` always evaluates to `X`. The line effectively assigns `clue.isSolved()` to `isSolved`.
- **Recommendation:** Consider changing the line `isSolved = isSolved.or(clue.isSolved());` to simply `isSolved = clue.isSolved();` in order to improve code clarity and remove a redundant logical operation.
- **Status:** Resolved.

3.37 | [INFO] Lack of progress report on transaction finalisation

- **Location(s):** UI
- **Description:** After creating the game and sending the transaction, there is no progress feedback to the user about waiting for transaction to be confirmed on frontend. only the below screen, which feels like it's just bugged.

[7] https://github.com/mina-builders-team/recursive-mastermind-zkApp/tree/99d4826994a7ddc9e7f3f5945b079e7facae6390

- **Recommendation:** Add information that user's tx is not yet included in a block and inform user to wait for transaction confirmation
- **Status:** Resolved.

3.38 | [INFO] Inconsistent error messaging for digit validation

- **Location(s):** zkApp/utils.ts#55-59
- **Description:** The error message gives the index of the erroneous digit instead of its actual value:

```
for (let i = 0; i < 4; i++) {
  this.digits[i]
    .greaterThan(7)
    .assertFalse(`Combination digit ${i + 1} is not in range [0, 7]!`);
}
```

- **Recommendation:** Consider changing the message to use the actual value.
- **Status:** Resolved.

4 | Test Coverage

4.1 | Test output

```
npm run test

(node:50540) ExperimentalWarning: VM Modules is an experimental feature and might
↪  change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
ts-jest[versions] (WARN) Version 5.8.2 of typescript installed has not been tested
↪  with ts-jest. If you're experiencing issues, consider using a supported version
↪  (>=4.3.0 <5.0.0-0). Please do not report issues in ts-jest if you are using
↪  unsupported versions.
PASS src/test/utils.test.ts
  utility.ts unit tests
    Combination class
      derive Combination from random numbers
         should create a valid Combination from an array of 4 digits in [1..7] (1
        ↪  ms)
         should throw an error if the array length is greater than 4 (9 ms)
         should throw an error if the array length is less than 4 (1 ms)
         should throw an error if the digits are not unique (4 ms)
         should throw an error if the digits are above from range [0, 7] (2 ms)
         should throw an error if the digits are not unique and above from range
        ↪  [0, 7]
      toBits() method
         should convert the combination to bits correctly for [1, 2, 3, 4]
         should convert the combination to bits correctly for [5, 6, 7, 1] (1 ms)
         should convert the combination to bits correctly for [7, 6, 5, 4]
         should convert the combination to bits correctly for [1, 2, 3, 7]
         should convert the combination to bits correctly for [2, 3, 4, 5]
         should convert the combination to bits correctly for [1, 2, 3, 6] (1 ms)
      compress() method
         should compress the combination correctly for [1, 2, 3, 4] (3 ms)
         should compress the combination correctly for [5, 6, 7, 1] (3 ms)
         should compress the combination correctly for [7, 6, 5, 4] (5 ms)
         should compress the combination correctly for [1, 2, 3, 7] (3 ms)
         should compress the combination correctly for [2, 3, 4, 5] (3 ms)
         should throw an error if a digit is greater than 7
         should throw an error if a digit is greater than 7 (1 ms)
      decompress() method
         should decompress the combination correctly for [1, 2, 3, 4]
         should decompress the combination correctly for [5, 6, 7, 1]
         should decompress the combination correctly for [7, 6, 5, 4] (1 ms)
         should decompress the combination correctly for [1, 2, 3, 7]
         should decompress the combination correctly for [2, 3, 4, 5]
      validate() method
         should validate a correct combination
         should throw an error for a combination with duplicate digits (1 ms)
         should throw an error for a combination with digits out of range
         should throw an error for a combination with digits out of range and
        ↪  duplicates
         should throw an error for a combination with digits out of range and
        ↪  duplicates
      updateHistory() method
         should update the history correctly for a new combination
```

```
      should update the history correctly for a new combination with existing
      ↪ history (1 ms)
  getElementFromHistory() method
      should return the correct combination from the history (1 ms)
      should return the correct combination from the history for the last
      ↪ element
      should return the correct combination from the history for the first
      ↪ element
      should return the correct combination from the history for an
      ↪ out-of-bounds index (1 ms)
      should return the correct combination with another random index
Clue class
  compress() method
      should compress the clue correctly for hits=2 and blows=1 (3 ms)
      should compress the clue correctly for hits=0 and blows=0 (4 ms)
      should compress the clue correctly for hits=3 and blows=2 (3 ms)
      should compress the clue correctly for hits=1 and blows=3 (3 ms)
      should compress the clue correctly for hits=0 and blows=4 (3 ms)
  decompress() method
      should decompress the clue correctly for hits=2 and blows=1
      should decompress the clue correctly for hits=0 and blows=0 (1 ms)
      should decompress the clue correctly for hits=3 and blows=2
      should decompress the clue correctly for hits=1 and blows=3 (1 ms)
      should decompress the clue correctly for hits=0 and blows=4
      should decompress the clue correctly for hits=4 and blows=0
  compress() and decompress() methods
      should compress and decompress the clue correctly for hits=2 and blows=1
      ↪ (1 ms)
      should compress and decompress the clue correctly for hits=0 and blows=0
      should compress and decompress the clue correctly for hits=3 and blows=1
  giveClue() method
      should give the correct clue for guess=[1, 2, 3, 4] and solution=[1, 2, 3,
      ↪ 4]
      should give the correct clue for guess=[1, 2, 3, 4] and solution=[4, 3, 2,
      ↪ 1]
      should give the correct clue for guess=[1, 2, 3, 4] and solution=[1, 3, 2,
      ↪ 4] (1 ms)
      should give the correct clue for guess=[1, 2, 3, 4] and solution=[2, 1, 4,
      ↪ 3]
      should give the correct clue for guess=[4, 7, 6, 1] and solution=[2, 1, 4,
      ↪ 3]
      should give the correct clue for guess=[3, 2, 6, 7] and solution=[2, 5, 3,
      ↪ 1]
      should give the correct clue for guess=[4, 2, 1, 7] and solution=[4, 5, 1,
      ↪ 3]
      should give the correct clue for guess=[5, 3, 1, 6] and solution=[4, 5, 1,
      ↪ 6]
  updateHistory() method
      should update the history correctly for a new clue (1 ms)
      should update the history correctly for a new clue with existing history
      ↪ (1 ms)
      should update the history correctly for a new clue with existing history
      ↪ (1 ms)
GameState class
  constructors
      should create a GameState object with default values
      should create a GameState object with custom values
  pack() method
```

```
        should pack the GameState object correctly (1 ms)
        should pack the GameState object correctly with default values (1 ms)
        should pack the GameState object correctly with custom values
        should pack the GameState object correctly with big values (1 ms)
      unpack() method
        should unpack the GameState object correctly (1 ms)
        should unpack the GameState object correctly with default values (1 ms)
        should unpack the GameState object correctly with custom values
        should unpack the GameState object correctly with big values (1 ms)

PASS src/test/testUtils.test.ts (12.02 s)
  Should generate StepProgramProof for given parameters
    Should generate codeMaster victory proof with random actions (4970 ms)
    Should generate codeBreaker victory proof with random actions (947 ms)
    Should generate an unsolved game proof with random actions. (1326 ms)
    Should generate codeMaster victory proof with predefined actions. (2332 ms)
    Should generate codeBreaker victory proof with predefined actions (985 ms)
    Should generate an unsolved game proof with predefined actions. (1293 ms)

PASS src/test/stepProgram.test.ts (5.957 s)
  Mastermind ZkProgram Tests
    Invalid game creation
      Invalid range of digits
        Should reject codeMaster with invalid secret combination: first digit is
        ↪  greater than 7 (11 ms)
        Should reject codeMaster with invalid secret combination: second digit is
        ↪  greater than 7
        Should reject codeMaster with invalid secret combination: third digit is
        ↪  greater than 7
        Should reject codeMaster with invalid secret combination: fourth digit is
        ↪  greater than 7
      Invalid uniqueness of digits
        Should reject codeMaster with invalid secret combination: second digit is
        ↪  not unique (4 ms)
        Should reject codeMaster with invalid secret combination: third digit is
        ↪  not unique
        Should reject codeMaster with invalid secret combination: fourth digit is
        ↪  not unique
        Should reject codeMaster with invalid secret combination: all digits are
        ↪  the same
    makeGuess method tests: first guess
      Should create a game successfully (2716 ms)
      Should reject giveClue in the wrong turn (6 ms)
      CodeBreaker should make a guess successfully (144 ms)
      Should reject makeGuess in the wrong turn (6 ms)
      Invalid guess
        Should reject codeBreaker with invalid guess combination: first digit is
        ↪  greater than 7 (1 ms)
        Should reject codeBreaker with invalid guess combination: second digit is
        ↪  greater than 7
        Should reject codeBreaker with invalid guess combination: third digit is
        ↪  greater than 7
        Should reject codeBreaker with invalid guess combination: fourth digit is
        ↪  greater than 7 (1 ms)
        Should reject codeBreaker with invalid guess combination: second digit is
        ↪  not unique
        Should reject codeBreaker with invalid guess combination: third digit is
        ↪  not unique
```

```
      Should reject codeBreaker with invalid guess combination: fourth digit is
      ↪  not unique
    Invalid code breaker signature
      Should reject codeBreaker with invalid signature: wrong public key (12 ms)
      Should reject codeBreaker with invalid signature: wrong message (10 ms)
      Should reject codeBreaker with invalid signature: wrong signer (11 ms)
  giveClue method tests
    Should reject any caller other than the codeMaster (7 ms)
    Should reject codeMaster with different salt (9 ms)
    Should reject codeMaster with non-compliant secret combination (7 ms)
    Should reject codeMaster with wrong secret combination and salt (7 ms)
    CodeMaster should give clue successfully (194 ms)
    Should reject the codeMaster from calling this method out of sequence (7 ms)
    Invalid code master signature
      Should reject codeMaster with invalid signature: wrong public key (7 ms)
      Should reject codeMaster with invalid signature: wrong message (10 ms)
      Should reject codeMaster with invalid signature: wrong signer (11 ms)
  Second guess
    Invalid code breaker signature
      Should reject codeBreaker with invalid signature: wrong public key (6 ms)
      Should reject codeBreaker with invalid signature: wrong message (10 ms)
      Should reject codeBreaker with invalid signature: wrong signer (9 ms)
    Make guess and try again
      Should accept valid guess (180 ms)
      Should reject the codebraker from calling this method out of sequence (5
      ↪  ms)
    Invalid code master signature
      Should reject codeMaster with invalid signature: wrong public key (7 ms)
      Should reject codeMaster with invalid signature: wrong message (11 ms)
      Should reject codeMaster with invalid signature: wrong signer (13 ms)
    Give clue and try again
      code Master should give clue successfully (161 ms)
      Should reject the codeMaster from calling this method out of sequence (6
      ↪  ms)
  Third guess
    Should accept valid guess (160 ms)
    Should reject the codebraker from calling this method out of sequence (5 ms)
    code Master should give clue successfully (170 ms)
    Should reject the codeMaster from calling this method out of sequence (6 ms)
  Fourth guess
    Should accept valid guess (141 ms)
    Should reject the codebraker from calling this method out of sequence (5 ms)
    code Master should give clue successfully (137 ms)
    Should reject the codeMaster from calling this method out of sequence (6 ms)
  Fifth guess
    Should accept valid guess (140 ms)
    Should reject the codebraker from calling this method out of sequence (5 ms)
    code Master should give clue successfully (168 ms)
    Should reject the codeMaster from calling this method out of sequence (6 ms)
  Sixth guess
    Should accept valid guess (150 ms)
    Should reject the codebraker from calling this method out of sequence (6 ms)
    code Master should give clue successfully (166 ms)
    Should reject the codeMaster from calling this method out of sequence (6 ms)
  Final guess
    Should accept valid guess (147 ms)
    Should give clue and report that the secret is solved (145 ms)
  Reject guess after completion
```

```
      Should reject next guess: secret is already solved (11 ms)
      Should reject next clue: secret is already solved (7 ms)
    New game after completion
      Should create a new game successfully with new secret (216 ms)
      Should solve the game in the first round (152 ms)
      Should give clue and report that the secret is solved (134 ms)
      Should reject next guess: secret is already solved (9 ms)
      should reject next clue: secret is already solved (6 ms)

PASS src/test/Mastermind.test.ts (42.677 s)
  Mastermind ZkApp Tests
    Deploy & Initialize Flow
      Deploy a Mastermind zkApp (285 ms)
      Initializes the game successfully (536 ms)
      Reject initializing the game again (64 ms)
    Reject other methods before initGame
      Reject calling acceptGame method call (89 ms)
      Reject calling submitGameProof method call (67 ms)
      Reject makeGuess method call (67 ms)
      Reject giveClue method call (80 ms)
    Reject invalid initGame calls
      Reject initGame if reward amount is less than 10 Mina: 0 Mina (72 ms)
      Reject initGame if reward amount is less than 10 Mina: 9.999.. Mina (66
      ↪  ms)
      Reject initGame with invalid secret: first digit is greater than 7 (69 ms)
      Reject initGame with invalid secret: second digit is greater than 7 (68
      ↪  ms)
      Reject initGame with invalid secret: third digit is greater than 7 (68 ms)
      Reject initGame with invalid secret: fourth digit is greater than 7 (69
      ↪  ms)
      Reject initGame with invalid secret: duplicate digits (77 ms)
      Should reject codeMaster with invalid secret combination: second digit is
      ↪  not unique (69 ms)
      Should reject codeMaster with invalid secret combination: third digit is
      ↪  not unique (98 ms)
      Should reject codeMaster with invalid secret combination: fourth digit is
      ↪  not unique (65 ms)
      Should reject codeMaster with invalid secret combination: all digits same
      ↪  and exceeding 7 (66 ms)
      Should reject codeMaster with invalid secret combination: all digits same
      ↪  (65 ms)
    Accepting a Game and Solve
      Accept the game successfully (325 ms)
      Reject accepting the game again (61 ms)
      Reject submitting a proof with wrong secret (61 ms)
      Reject claiming reward before solving (69 ms)
      Reject reward claim from intruder (68 ms)
      Submit with correct game proof and wrong winner pubkey (227 ms)
      Reject submitting the same proof again (62 ms)
      Claim reward successfully (322 ms)
      Reject submitting after reward claim (60 ms)
    Reject dependent methods before acceptGame
      Reject submitGameProof before acceptGame (60 ms)
      Reject forfeitWin before acceptGame (74 ms)
      Reject makeGuess before acceptGame (59 ms)
      Reject giveClue before acceptGame (68 ms)
    codeMaster reimbursed reward
      codeMaster reimbursed reward succesfully (318 ms)
```

```
      Reject accepting the game after reimbursed (62 ms)
   Submitting Correct Game Proof and Claiming Reward
      Submit with partial game proof (206 ms)
      Reject submitting the same partial proof again (60 ms)
      Submit with correct game proof with wrong winner (197 ms)
      Reject submitting the same proof again (58 ms)
      Claim reward (321 ms)
      Reject forfeitWin method call after finalization (73 ms)
   Code Breaker punished for timeout
      Reject forfeitWin method call of intruder (73 ms)
      Reject forfeitWin method call of codeMaster (74 ms)
      Reject forfeitWin method call for random player (73 ms)
      Penalty for codeBreaker (328 ms)
      Reject forfeitWin method call again (76 ms)
   Code Master punished for timeout
      Penalty for codeMaster (327 ms)
      Reject forfeitWin method call again (72 ms)
   Code Master wins
      Intruder tries to make guess before code creaker (66 ms)
      makeGuess method (273 ms)
      Intruder tries to give clue (75 ms)
      giveClue method (269 ms)
      Intruder tries to make guess again (65 ms)
      Claim reward successfully (320 ms)
   StepProgramProof settlements tests
      Should generate a proof with randomly chosen actions for codeMaster victory
      ↪  and settle. (3109 ms)
      Should generate a proof with predefined actions for codeMaster victory and
      ↪  settle. (3132 ms)
      Should generate a proof with randomly chosen actions for codeBreaker victory
      ↪  and settle. (2044 ms)
      Should generate a proof with predefined actions for codeBreaker victory and
      ↪  settle. (2163 ms)
      Should generate a proof with randomly chosen actions for unsolved game.
      ↪  (2054 ms)
   Submitting Invalid Game Proofs
      Reject submitting a proof with intruder code breaker (493 ms)
      Reject submitting a proof with intruder code master (484 ms)
      Reject submitting a proof with wrong solution hash: wrong secret (473 ms)
      Reject submitting a proof with wrong solution hash: wrong salt (508 ms)
      Submit a proof with 3rd round then try to submit another one with round 2
      ↪  (1942 ms)
   Recovery if offchain recursion is not available
      Code breaker should be able to start game with makeGuess (269 ms)
      Code master should be able to continue game with stepProgramProof with 3rd
      ↪  round (1188 ms)
      Code master wait more than PER_TURN_GAME_DURATION and rejected to make guess
      ↪  onchain (65 ms)
      Settle stepProgramProof with unsolved game (1770 ms)
      Intruder tries to make guess and fails (66 ms)
      Code breaker should be able to continue game with makeGuess (261 ms)
      Intruder tries to give clue and fails (73 ms)
      Code master should be able to continue game with giveClue (278 ms)
      Code breaker should be able to continue game with makeGuess (275 ms)
      Code master should be able to continue game with giveClue and win (278 ms)
      Code breaker should not be able to continue game with makeGuess (59 ms)
      Code master should be able to claim reward (400 ms)
   Play it totally onchain with maximum duration
```

```
        Turn 1 (540 ms)
        Turn 2 (528 ms)
        Turn 3 (525 ms)
        Turn 4 (523 ms)
        Turn 5 (537 ms)
        Turn 6 (566 ms)
        Turn 7 (533 ms)
        Claim reward (337 ms)

Test Suites: 4 passed, 4 total
Tests:       226 passed, 226 total
Snapshots:   0 total
Time:        62.124 s
Ran all test suites.
```

A | Disclaimers

The audit makes no statements or warranty about utility of the code, safety of the code, suitability of the business model, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purpose

A.1 | Client Confidentiality

This document contains Client Confidential information and may not be copied without written permission.

A.2 | Proprietary Information

The content of this document should be considered proprietary information. Extropy gives permission to copy this report for the purposes of disseminating information within your organisation or any regulatory agency.

B | Automated Tools results

B.1 | Burp Suite results

Penetration testing using Burp Suite was conducted on 21/7/25
No significant issues were found.

C | Game Ending Analysis

We created truth tables for the combinations of variables that are involved to determining the end of a game.
For example in submitGameProof
A=1 "Codemaster is marked as winner"
A=0 "Codebreaker is marked as winner"
B=1 "Is solved"
B=0 "Is not solved"
C=1 "Turns less than 16"
C=0 "Turns less than or equal to 16"

Intended	Code	A	B	C	Description
0	0	0	0	0	"Codebreaker is marked as winner" AND "Is not solved" AND "Turns ¿= 16"
0	0	0	0	1	"Codebreaker is marked as winner" AND "Is not solved" AND "Turns ¡ 16"
0	0	0	1	0	"Codebreaker is marked as winner" AND "Is solved" AND "Turns ¿= 16"
1	1	0	1	1	"Codebreaker is marked as winner" AND "Is solved" AND "Turns ¡ 16"
1	1	1	0	0	"Codemaster is marked as winner" AND "Is not solved" AND "Turns ¿= 16"
0	0	1	0	1	"Codemaster is marked as winner" AND "Is not solved" AND "Turns ¡ 16"
1	0	1	1	0	"Codemaster is marked as winner" AND "Is solved" AND "Turns ¿= 16"
0	0	1	1	1	"Codemaster is marked as winner" AND "Is solved" AND "Turns ¡ 16"

The discrepancy we found between intended and code was one that we didn't think could arise because of checking done elsewhere in the code.

D | Decision Tree

A decision tree can be simply created to guide a codebreaker to an optimal strategy, a example snippet is given here. These can be produced for any combination of turns and number of digits.

- **metadata**
  - **digits:** 0, 1, 2, 3, 4, 5, 6, 7
  - **total_combinations:** 1680
  - **max_attempts:** 6
- **tree**
  - **initial_guess:** [0, 1, 2, 3]
  - **responses:**
    - **(4,0)** → **solution:** [0, 1, 2, 3]
    - **(3,0)** →
      - **next_guess:** [0, 1, 4, 5]
      - **responses:**
      - **(2,1)** →
      - **next_guess:** [0, 1, 2, 4]
      - **responses:**
      - **(2,0)** → **solution:** [0, 1, 5, 3]

extropy

